



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Processamento de Linguagens

Ano Letivo de 2025/2026

Trabalho Prático - Grupo 24

Afonso Martins
a106931

Guilherme Costa
a107326

Gonçalo Castro
a107337

Maio, 2026

PL

Índice

1. Introdução	1
2. Arquitetura Geral do Compilador	1
2.1. Decisões de Design	2
3. Análise Léxica	2
3.1. Tokens reconhecidos	2
3.2. <i>Case insensitivity</i>	2
3.3. Literais	3
3.4. Operadores relacionais e lógicos	3
3.5. Comentários e espaços em branco	3
3.6. Prioridade das regras	3
4. Análise Sintática	4
4.1. Gramática implementada	4
4.2. Precedência de operadores	5
4.3. Construção da AST	6
4.4. Suporte a subprogramas	6
4.5. Tratamento de erros sintáticos	7
5. Análise Semântica	7
5.1. Estrutura da tabela de símbolos	7
5.2. Passagens de análise	8
5.3. Verificações implementadas	8
5.3.1. Redecaração de variáveis	8
5.3.2. Uso de variáveis não declaradas	8
5.3.3. Validação de labels	8
5.3.4. Validação de chamadas a subprogramas	8
5.3.5. Variáveis declaradas mas não utilizadas	8
5.3.6. Verificação de tipos em expressões	8
6. Geração de Código	9
6.1. Modelo de execução da VM	9
6.2. Alocação de variáveis	9
6.3. Geração por construção	9
6.3.1. Atribuição	9
6.3.2. Expressões aritméticas	10
6.3.3. Menos unário (-)	10
6.3.4. Operadores relacionais e lógicos	10
6.3.5. Estrutura condicional	10
6.3.6. Ciclo DO	11
6.3.7. Entrada e saída (I/O)	11
6.3.8. Subprogramas e frames de ativação	11
6.3.8.1. Convenção de chamada	11
6.3.8.2. Exemplo — FUNCTION com frame de ativação	12
7. Executar o Compilador	12
7.1. Instalação (Primeira Vez)	12
7.2. Compilar um Programa	12
7.2.1. Exemplos Rápidos	13
7.3. Como Testar Tudo	13
7.3.1. Rodar Todos os Testes	13
7.3.2. Testes por Módulo	13

8. Conclusões e Trabalho Futuro 13

1. Introdução

O Fortran (*Formula Translation*) foi a primeira linguagem de programação de alto nível com adoção generalizada, surgindo na década de 1950 como resposta à necessidade de expressar cálculo científico de forma mais próxima da notação matemática. A sua versão de 1977 — Fortran 77, padronizada pelo ANSI sob a norma X3.9-1978 — consolidou um conjunto estável de construções que continuam a ser relevantes no estudo de técnicas de compilação, nomeadamente pela simplicidade da gramática, pela tipagem estática, e pelo uso explícito de labels e instruções de controlo de fluxo como GOTO e DO.

O presente projeto tem como objetivo a construção de um compilador funcional para um subconjunto representativo do Fortran 77, capaz de traduzir programas válidos em código executável pela máquina virtual académica disponibilizada na disciplina. O trabalho cobre as quatro etapas clássicas de um compilador de estrutura simples:

1. **Análise léxica** — decomposição do código fonte numa sequência de *tokens*;
2. **Análise sintática** — validação da estrutura gramatical e construção de uma Árvore de Sintaxe Abstrata (AST);
3. **Análise semântica** — verificação de tipos, declaração de variáveis e coerência de *labels*;
4. **Geração de código** — tradução da AST para instruções da VM.

O compilador foi implementado em Python, tirando partido da biblioteca PLY (*Python Lex-Yacc*), tal como sugerido no decorrer da UC.

2. Arquitetura Geral do Compilador

O compilador desenvolvido segue uma arquitetura modular, organizada em várias fases bem definidas do processo de compilação. Cada fase é implementada num módulo independente, promovendo separação de responsabilidades e facilidade de manutenção.

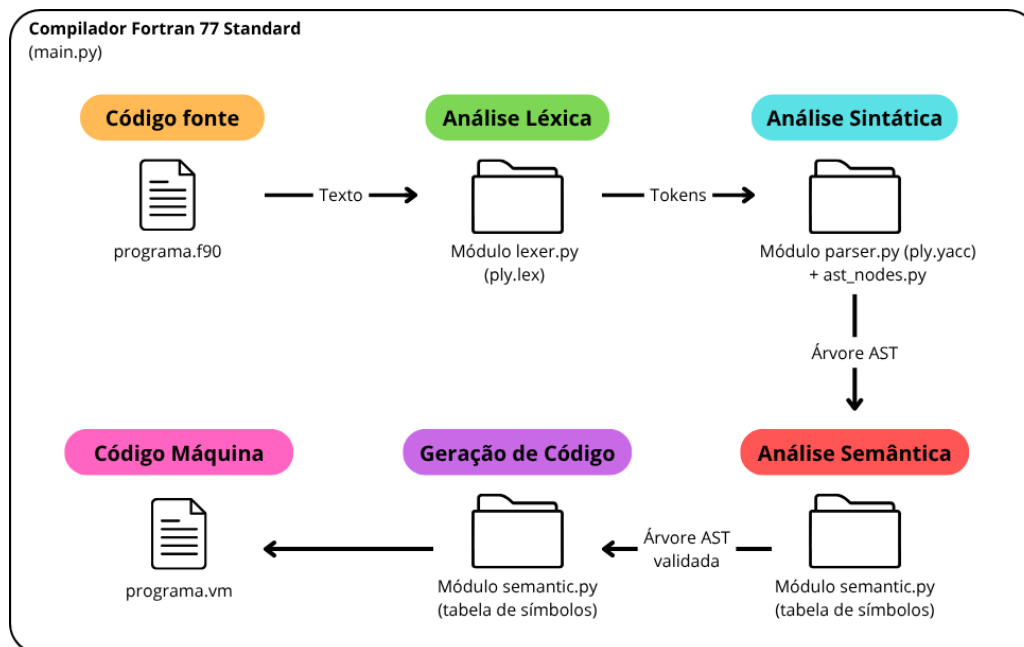


Figura 1: Diagrama da Pipeline de funcionamento geral do Compilador desenvolvido

Os principais módulos são:

- **lexer.py**: responsável pela análise léxica, convertendo o código fonte em *tokens*.
- **parser.py**: responsável pela análise sintática, construindo a AST (*Abstract Syntax Tree*).
- **semantic.py**: realiza análise semântica e validações (ex: tipos, variáveis declaradas).
- **codegen.py**: gera código para uma máquina virtual baseada em *stack*.

- `ast_nodes.py`: define as estruturas de dados da AST.
- `main.py`: interface de linha de comandos que coordena todo o pipeline.

O compilador segue um pipeline sequencial, onde a saída de cada fase serve de entrada para a seguinte.

2.1. Decisões de Design

Foi utilizada a biblioteca PLY (Python Lex-Yacc) para implementar o *lexer* e o *parser*, permitindo definir a gramática de forma declarativa e gerar automaticamente um **parser LALR(1)**.

Optou-se por suportar sintaxe **free-form**, ao contrário do Fortran 77 clássico (fixed-form), simplificando a implementação e tornando o sistema mais flexível.

A interface de utilização é feita através de linha de comandos (`main.py`), permitindo compilar ficheiros fonte e gerar código VM de forma simples.

3. Análise Léxica

A análise léxica é responsável por transformar o código fonte numa sequência de *tokens*, que serão posteriormente consumidos pelo *parser*. O *lexer* foi implementado com PLY (`ply.lex`), em que cada tipo de *token* é definido por uma expressão regular sob a forma de função Python ou de variável de *string*, conforme a complexidade da regra.

3.1. Tokens reconhecidos

O compilador reconhece os seguintes tipos de *tokens*, agrupados por categoria:

Categoria	Tokens
Identificadores	IDENTIFIER
Literais inteiros	NUMBER — ex: 42
Literais reais	REAL_NUMBER — ex: 3.14, 1.5e-3
Literais string	STRING — plicas '...' ou aspas "..."
Literais booleanos	TRUE (.TRUE.), FALSE (.FALSE.)
Palavras reservadas	PROGRAM, END, INTEGER, REAL, LOGICAL, IF, THEN, ELSE, ENDIF, DO, CONTINUE, GOTO, PRINT, READ, SUBROUTINE, FUNCTION, CALL, RETURN
Operadores aritméticos	PLUS (+), MINUS (-), TIMES (*), DIVIDE (/)
Operadores relacionais	EQ (.EQ.), NE (.NE.), LT (.LT.), LE (.LE.), GT (.GT.), GE (.GE.)
Operadores lógicos	AND (.AND.), OR (.OR.), NOT (.NOT.)
Delimitadores	LPAREN ((), RPAREN ()), COMMA (,)
Atribuição	ASSIGN (=)

3.2. Case insensitivity

O Fortran 77 é *case-insensitive* — PROGRAM, program e Program são equivalentes, e X e x referem-se à mesma variável. Esta característica é implementada na regra de identificadores, onde o valor lido é convertido para minúsculas antes de ser pesquisado no dicionário de palavras reservadas:

```
def t_IDENTIFIER(self, t):
    r"[a-zA-Z][a-zA-Z0-9_]*"
```

```
t.type = self.reserved.get(t.value.lower(), "IDENTIFIER")
return t
```

O valor do *token* (`t.value`) é preservado com o *case* original — apenas a classificação do tipo usa a forma em minúsculas. Assim, uma variável declarada como `INTEGER MyVar` é armazenada na tabela de símbolos como `"MyVar"`.

3.3. Literais

São suportados quatro tipos de literais:

- **Inteiros** — sequência de dígitos decimais: 42, 0, 1000.
- **Reais** — com ponto decimal ou notação científica: 3.14, .5, 1.5e-3. A regra `t_REAL_NUMBER` é definida antes de `t_NUMBER`, garantindo que 3.14 seja reconhecido como `REAL_NUMBER` e não como dois tokens separados.
- **Strings** — delimitadas por plicas ('Ola, Mundo!') ou aspas ("hello"). As aspas externas são removidas do valor do *token*.
- **Booleanos** — `.TRUE.` e `.FALSE.`, convertidos para `True/False` Python.

3.4. Operadores relacionais e lógicos

O Fortran 77 utiliza operadores delimitados por pontos, distintos dos símbolos comuns noutras linguagens. Por exemplo, `.EQ.` corresponde a `==` em C. Estes operadores são reconhecidos por funções dedicadas (e.g., `t_EQ`, `t_AND`), que têm prioridade sobre a regra genérica de identificadores no PLY. Cada função aceita tanto maiúsculas como minúsculas em qualquer posição:

```
def t_EQ(self, t):
    r"\.[Ee][Qq]\."
    return t

def t_AND(self, t):
    r"\.[Aa][Nn][Dd]\."
    return t
```

3.5. Comentários e espaços em branco

Comentários são introduzidos pelo carácter `!` e estendem-se até ao fim da linha. A regra `t_COMMENT` consome a sequência sem devolver nenhum *token*, tornando os comentários invisíveis para o *parser*:

```
def t_COMMENT(self, t):
    r"!.*"
    pass
```

Espaços, tabulações e retornos de carro são declarados em `t_ignore`:

```
t_ignore = " \t\r"
```

Quebras de linha são tratadas por `t_newline`, que incrementa o contador de linha interno do *lexer* (`t.lexer.lineno`) para que as mensagens de erro reportem a linha correta.

3.6. Prioridade das regras

O PLY determina a prioridade das regras da seguinte forma: funções têm prioridade sobre variáveis de string, e entre funções a ordem é a de declaração no ficheiro. Isto é relevante em dois pontos:

- `t_REAL_NUMBER` é declarada antes de `t_NUMBER`, garantindo que 3.14 não seja tokenizado como 3, ., 14.
- As funções dos operadores com pontos (`.EQ.`, `.AND.`, etc.) são declaradas antes de `t_IDENTIFIER`, evitando que `.EQ.` seja lido como . seguido do identificador `EQ` seguido de ..

4. Análise Sintática

A análise sintática verifica se a sequência de *tokens* produzida pelo *lexer* respeita a gramática da linguagem, construindo simultaneamente uma representação estruturada do programa sob a forma de uma *Abstract Syntax Tree* (AST). O *parser* foi implementado com PLY (ply.yacc), que gera automaticamente um *parser* LALR(1) a partir de regras de produção definidas como funções Python.

4.1. Gramática implementada

Cada função `p_*` no módulo `parser.py` corresponde a uma ou mais produções. A gramática completa do subconjunto suportado é a seguinte:

```
program          : program_units

program_units    : program_unit
                  | program_units program_unit

program_unit     : main_program
                  | subroutine
                  | function

main_program     : PROGRAM IDENTIFIER statements END
                  | PROGRAM IDENTIFIER END

statements       : statement
                  | statements statement

statement        : declaration | assignment | print_stmt | read_stmt
                  | if_stmt | do_loop | goto_stmt | call_stmt | return_stmt

declaration      : type_spec var_list
type_spec        : INTEGER | REAL | LOGICAL
var_list         : IDENTIFIER
                  | var_list COMMA IDENTIFIER

assignment       : IDENTIFIER ASSIGN expression

print_stmt       : PRINT TIMES print_list
print_list       : COMMA expression
                  | print_list COMMA expression

read_stmt        : READ TIMES read_list
read_list        : COMMA IDENTIFIER
                  | read_list COMMA IDENTIFIER

if_stmt          : IF LPAREN expression RPAREN THEN statements ENDIF
                  | IF LPAREN expression RPAREN THEN statements ELSE
                    statements ENDIF

do_loop          : DO NUMBER IDENTIFIER ASSIGN expression COMMA expression
                  statements NUMBER CONTINUE

goto_stmt        : GOTO NUMBER
call_stmt        : CALL IDENTIFIER LPAREN arg_list RPAREN
                  | CALL IDENTIFIER
return_stmt      : RETURN

expression       : expr_or
expr_or          : expr_and | expr_or OR expr_and
expr_and         : expr_not | expr_and AND expr_not
```

```

expr_not      : expr_rel | NOT expr_not
expr_rel      : expr_arith
               | expr_arith EQ  expr_arith | expr_arith NE expr_arith
               | expr_arith LT  expr_arith | expr_arith LE expr_arith
               | expr_arith GT  expr_arith | expr_arith GE expr_arith

expr_arith    : term
               | expr_arith PLUS term | expr_arith MINUS term

term          : factor
               | term TIMES factor | term DIVIDE factor

factor        : primary
               | MINUS factor %prec UMINUS

primary       : NUMBER | REAL_NUMBER | STRING | TRUE | FALSE
               | IDENTIFIER
               | IDENTIFIER LPAREN arg_list RPAREN
               | LPAREN expression RPAREN

arg_list      : expression | arg_list COMMA expression

subroutine    : SUBROUTINE IDENTIFIER LPAREN param_list RPAREN
               statements END
               | SUBROUTINE IDENTIFIER statements END

function      : type_spec FUNCTION IDENTIFIER LPAREN param_list RPAREN
               statements END
               | type_spec FUNCTION IDENTIFIER statements END

param_list    : IDENTIFIER | param_list COMMA IDENTIFIER

```

4.2. Precedência de operadores

A precedência e associatividade foram declaradas explicitamente na tabela precedence do *parser*, da menor para a maior prioridade:

Nível	Associação	Operadores
1 (menor)	esquerda	.OR.
2	esquerda	.AND.
3	direita	.NOT.
4	esquerda	.EQ. .NE. .LT. .LE. .GT. .GE.
5	esquerda	+ -
6	esquerda	* /
7 (maior)	direita	Unário - (UMINUS)

O nível 7 (UMINUS) é uma *pseudo-precedência* declarada apenas para resolver a ambiguidade do menos unário. A regra `factor : MINUS factor` usa a diretiva `%prec UMINUS`, forçando o PLY a tratar o - nesse contexto como operador unário de alta prioridade, distinto do - binário do nível 5. Sem esta declaração, uma expressão como `-2 * 3` poderia ser mal interpretada.

Exemplos de expressões e a sua árvore resultante:

- `1 + 2 * 3` $\rightarrow$ `BinaryOp(+, IntLiteral(1), BinaryOp(*, IntLiteral(2), IntLiteral(3)))`
multiplicação tem prioridade sobre adição
- `A .OR. B .AND. C` $\rightarrow$ `BinaryOp(.OR., Variable(A), BinaryOp(.AND., Variable(B), Variable(C)))`
.AND. tem prioridade sobre .OR.
- `-X + 1` $\rightarrow$ `BinaryOp(+, UnaryOp(-, Variable(X)), IntLiteral(1))`
unário tem prioridade sobre binário

4.3. Construção da AST

Durante a análise sintática é construída explicitamente uma AST, usando as classes definidas em `ast_nodes.py`. Cada regra de produção cria e devolve o nó correspondente. A hierarquia principal é:

Classe	Representa
Program	Bloco PROGRAM ... END
Declaration	INTEGER X, Y
Assignment	X = expr
PrintStatement	PRINT *, ...
ReadStatement	READ *, ...
IfStatement	IF (...) THEN ... [ELSE ...] ENDIF
DoLoop	DO label I = start, end ... label CONTINUE
GotoStatement	GOTO label
ContinueStatement	label CONTINUE
Subroutine	SUBROUTINE nome(...) ... END
Function	type FUNCTION nome(...) ... END
CallStatement	CALL nome(...)
ReturnStatement	RETURN
BinaryOp	Operação binária: left op right
UnaryOp	Operação unária: op operand
Variable	Referência a variável: X
IntLiteral	Inteiro: 42
RealLiteral	Real: 3.14
StringLiteral	String: 'hello'
BooleanLiteral	.TRUE. ou .FALSE.
FunctionCall	Chamada de função em expressão: F(x)

Por exemplo, a instrução `X = 1 + 2 * 3` produz a subárvore:

```
Assignment
  target: 'X'
  value:
    BinaryOp(op='+')
      left: IntLiteral(1)
      right: BinaryOp(op='*')
        left: IntLiteral(2)
        right: IntLiteral(3)
```

4.4. Suporte a subprogramas

Para além do programa principal, a gramática suporta a definição de subprogramas (SUBROUTINE e FUNCTION) como unidades independentes no mesmo ficheiro. A regra `program_units` permite que um ficheiro contenha múltiplas unidades; o *parser* identifica o nó Program (programa principal) e devolve-o como raiz da AST:

```
def p_program(self, p):
    """program : program_units"""
    units = p[1]
    main = next((u for u in units if isinstance(u, Program)), None)
    subprograms = [u for u in units if not isinstance(u, Program)]
    if main is not None:
        main.statements = main.statements + subprograms
        p[0] = main
    else:
        p[0] = units[0] if len(units) == 1 else units
```

Os nós Subroutine e Function são incorporados diretamente em `Program.statements`, de modo a que o parser devolva sempre um único nó `Program` como raiz da AST. O gerador de código emite os subprogramas após o `STOP` do programa principal durante a travessia de `statements`.

4.5. Tratamento de erros sintáticos

Quando o *parser* encontra um *token* inesperado, é lançada uma exceção `ParserError` com o número de linha e o valor do *token* que causou o erro:

```
def p_error(self, p):
    if p:
        raise ParserError(
            f"Erro sintático em '{p.value}'",
            lineno=p.lineno,
            value=p.value,
        )
    else:
        raise ParserError("Erro sintático no fim do input")
```

Não foi implementada recuperação de erros (error token do PLY), pelo que o compilador para na primeira ocorrência. Esta é uma limitação conhecida — uma extensão futura poderia usar `p_error` com sincronização por tokens de continuação para reportar múltiplos erros numa única passagem.

5. Análise Semântica

A análise semântica valida o significado do programa após a análise sintática, garantindo que as construções são logicamente corretas além de estruturalmente válidas. Esta fase foi implementada no módulo `semantic.py`, através da classe `SemanticAnalyzer`, que percorre a AST em múltiplas passagens.

5.1. Estrutura da tabela de símbolos

Cada variável ou subprograma declarado é representado por um objeto `Symbol` com os seguintes campos:

Campo	Tipo	Descrição
name	str	Nome da variável
type_spec	str	INTEGER, REAL, LOGICAL, SUBROUTINE, PARAM
line	int	Linha de declaração
used	bool	Se foi referenciada em alguma expressão
assigned	bool	Se foi alvo de atribuição ou leitura

A tabela é um dicionário `Dict[str, Symbol]` que mapeia nomes (com o *case* original) para os respetivos símbolos.

5.2. Passagens de análise

A análise é feita em cinco passagens sequenciais sobre a AST, separando responsabilidades e garantindo que a informação necessária está disponível em cada etapa:

1. **_collect_declarations** — percorre todos os nós Declaration, Subroutine e Function, preenchendo a tabela de símbolos. Os parâmetros formais de subprogramas são inseridos com tipo PARAM; quando a declaração de tipo explícita do parâmetro for encontrada no corpo, o tipo é atualizado:

```
if self.symbols[var_name].type_spec == "PARAM":
    self.symbols[var_name].type_spec = node.type_spec
```

1. **_collect_labels** — recolhe todos os labels numéricos em três conjuntos distintos: labels (labels de CONTINUE), do_labels (labels de DO) e goto_labels (labels referenciados por GOTO). A separação permite validar cada caso de forma independente.
2. **_validate_statements** — percorre todos os *statements* verificando uso de variáveis não declaradas, marcando variáveis como used e assigned.
3. **_check_unused_variables** — gera avisos (*warnings*) para variáveis declaradas mas nunca referenciadas. Avisos não bloqueiam a compilação.
4. **_validate_goto_labels** — cruza os três conjuntos de labels para detetar:
 - GOTO para label inexistente
 - DO sem CONTINUE correspondente

5.3. Verificações implementadas

5.3.1. Redecaração de variáveis

É verificado se uma variável é declarada mais do que uma vez no mesmo âmbito. A exceção é a redeclaração de um parâmetro com tipo explícito, que é permitida e utilizada para especializar o tipo genérico PARAM:

```
INTEGER FUNCTION CONVRT(N, B)
    INTEGER N, B    # redeclaração válida: PARAM -> INTEGER
```

5.3.2. Uso de variáveis não declaradas

Qualquer referência a uma variável (em expressões, atribuições, READ, variável de controlo de DO) que não conste na tabela de símbolos gera um erro:

```
X = 5    # ERRO se X não foi declarado
```

5.3.3. Validação de labels

Os labels são validados em duas direções:

- Todo o GOTO N exige que exista um N CONTINUE ou um DO N no programa.
- Todo o DO N exige que exista um N CONTINUE correspondente.

Labels duplicados (dois CONTINUE com o mesmo número) são também detetados na segunda passagem.

5.3.4. Validação de chamadas a subprogramas

Tanto CALL nome(...) como chamadas de função em expressões (FunctionCall) verificam se nome está na tabela de símbolos com tipo SUBROUTINE ou o tipo de retorno da função.

5.3.5. Variáveis declaradas mas não utilizadas

Gera um aviso (não erro) para cada variável cujo campo used permaneça False após a análise. Isto ajuda a identificar declarações redundantes sem bloquear a compilação.

5.3.6. Verificação de tipos em expressões

O analisador semântico verifica a compatibilidade de tipos nas expressões e nas atribuições, detetando usos incorretos de LOGICAL em contexto aritmético e misturas de tipos incompatíveis. A inferência de tipos é feita pelo método _get_expr_type, que devolve "INTEGER", "REAL", "LOGICAL" ou None (tipo desconhecido/erro) para qualquer expressão.

As regras aplicadas são:

Contexto	Situação	Resultado
Ops. aritméticas + - * /	Operando LOGICAL	Erro — LOGICAL não pode ser usado em contexto aritmético
Ops. aritméticas + - * /	Mistura INTEGER e REAL	Aviso — possível perda de precisão
.AND. .OR.	Operando não-LOGICAL	Erro — operação lógica exige LOGICAL
.NOT.	Operando não-LOGICAL	Erro — .NOT. exige LOGICAL
Negação unária -	Operando LOGICAL	Erro — não se pode negar aritmeticamente um LOGICAL
.LT. .LE. .GT. .GE.	Operando LOGICAL	Erro — LOGICAL não tem ordem definida
Atribuição	LOGICAL atribuído a INTEGER ou REAL	Erro — tipos incompatíveis
Atribuição	INTEGER ou REAL atribuído a LOGICAL	Erro — tipos incompatíveis
Atribuição	REAL atribuído a INTEGER	Aviso — possível truncamento

6. Geração de Código

A geração de código converte a AST validada em instruções para a máquina virtual *stack-based* fornecida na disciplina. Esta fase está implementada no módulo `codegen.py`, através da classe `CodeGenerator`.

6.1. Modelo de execução da VM

A máquina virtual opera com uma *stack* de operandos e uma região de memória global para variáveis. O modelo de execução é:

- valores são empilhados com instruções `PUSH*`;
- operações aritméticas, lógicas e relacionais consomem operandos do topo e empilham o resultado;

6.2. Alocação de variáveis

O gerador distingue dois tipos de variáveis:

- **Globais** — declaradas no corpo do programa principal; acedidas com `PUSHG/STOREG`. Alocadas com `PUSHN` antes de `START`.
- **Locais** — declaradas dentro de subroutines ou functions; acedidas com `PUSHL/STOREL` no frame de ativação correspondente.

A separação é feita em `_collect_globals`, que para na fronteira dos subprogramas, e `_build_local_frame`, que constrói o mapa de índices locais para cada subprograma.

```
PUSHN 3    # reserva espaço para 3 variáveis globais
START
```

6.3. Geração por construção

6.3.1. Atribuição

A expressão é avaliada (empilhando o resultado) e depois armazenada:

```
X = 5 + 3
```

Gera:

```

PUSHI 5
PUSHI 3
ADD
STOREG 0    # endereço de X

```

6.3.2. Expressões aritméticas

As expressões são geradas por **travessia recursiva pós-ordem** da AST, garantindo que os operandos são empilhados antes da operação:

$$X = (1 + 2) * 3$$

Gera:

```

PUSHI 1
PUSHI 2
ADD
PUSHI 3
MUL
STOREG 0

```

6.3.3. Menos unário (-)

O menos unário não tem instrução direta na VM. É implementado com SWAP/SUB:

```

X = -Y
PUSHG 1    # Y
PUSHI 0
SWAP
SUB        # 0 - Y
STOREG 0

```

6.3.4. Operadores relacionais e lógicos

Os operadores Fortran são mapeados para as instruções equivalentes da VM:

Operador Fortran	Instruções VM
.EQ.	EQUAL
.NE.	EQUAL + NOT
.LT.	INF
.LE.	INFEQ
.GT.	SUP
.GE.	SUPEQ
.AND.	AND
.OR.	OR
.NOT.	NOT

6.3.5. Estrutura condicional

O IF-THEN-ELSE-ENDIF é compilado com dois labels gerados dinamicamente (ELSE_n e ENDIF_n), garantindo que não colidem com os labels numéricos do Fortran:

```

IF (X .GT. 0) THEN
    X = X + 1
ELSE
    X = X - 1
ENDIF

```

Gera:

```

PUSHG 0    # X
PUSHI 0

```

```

SUP                # X > 0
JZ ELSE1
PUSHG 0
PUSHI 1
ADD
STOREG 0
JUMP ENDIF2
ELSE1:
PUSHG 0
PUSHI 1
SUB
STOREG 0
ENDIF2:

```

6.3.6. Ciclo DO

O DO é compilado como um ciclo com verificação de condição no início (INFEQ) e incremento explícito da variável de controlo:

```

DO 10 I = 1, 5
    PRINT *, I
10 CONTINUE

```

Gera:

```

PUSHI 1
STOREG 0        # I = 1
LOOP1:
PUSHG 0        # I
PUSHI 5
INFEQ          # I <= 5
JZ ENDL00P2
PUSHG 0
WRITEI
Writeln
PUSHG 0        # incremento no CONTINUE
PUSHI 1
ADD
STOREG 0
JUMP LOOP1
ENDL00P2:

```

6.3.7. Entrada e saída (I/O)

PRINT emite uma instrução de escrita por expressão, seguida de WRITELN. A instrução de escrita é escolhida pelo tipo real da expressão:

- StringLiteral → WRITES
- RealLiteral → WRITEF
- Variable → consulta `symbol_table.get_type(nome)`: se REAL emite WRITEF, caso contrário WRITEI
- Restantes expressões → WRITEI

READ lê sempre como string e converte com ATOF para variáveis REAL ou atoi para INTEGER/LOGICAL, consultando o tipo na tabela de símbolos.

6.3.8. Subprogramas e frames de ativação

Os subprogramas (SUBROUTINE e FUNCTION) são emitidos após o STOP do programa principal. A passagem de argumentos e o acesso a variáveis locais são feitos através de **frames de ativação**, isolando completamente o estado de cada chamada do espaço global.

6.3.8.1. Convenção de chamada

1. O chamador empilha os argumentos em ordem, sem escrever em variáveis globais.
2. PUSHA nome / CALL transfere o controlo e cria um novo frame.
3. Dentro do subprograma, os parâmetros estão em PUSHL 0, PUSHL 1, ...; as variáveis locais declaradas seguem nos índices seguintes, alocadas com PUSHN.

4. Para FUNCTION, o nome da função é tratado como uma variável local extra (último slot do frame): o corpo atribui-lhe o resultado e, antes do RETURN, o slot é empurrado para a stack.
5. RETURN restaura o frame anterior; para funções, o valor de retorno fica no topo da stack.

6.3.8.2. Exemplo — FUNCTION com frame de ativação

```
INTEGER FUNCTION ADD(A, B)
  INTEGER A, B
  ADD = A + B
END
```

Gera:

```
ADD:
  PUSHN 1      # slot para variável de retorno (ADD)
  PUSHL 0      # A
  PUSHL 1      # B
  ADD
  STOREL 2     # ADD = A + B
  PUSHL 2     # empurra valor de retorno antes do RETURN
  RETURN
```

No lado do *caller*, os argumentos são empilhados antes de PUSHA/CALL; após o retorno, o valor de retorno está no topo da *stack* e pode ser usado diretamente pela expressão que invocou a função.

Esta abordagem elimina conflitos de nomes entre variáveis locais e globais e torna as chamadas seguras para recursão.

7. Executar o Compilador

7.1. Instalação (Primeira Vez)

```
# Entrar na pasta base do repositório
cd PL_Grupo24

# Criar ambiente virtual
python3 -m venv venv

# Ativar (Linux)
source venv/bin/activate

# Ativar ambiente virtual (Windows CMD)
venv\Scripts\activate.bat

# Instalar dependências
pip install -r requirements.txt
```

7.2. Compilar um Programa

```
# Compilação básica (gera .vm)
python main.py programa.f90

# Ver apenas tokens (tokenização)
python main.py programa.f90 --tokenize

# Ver árvore sintática (AST)
python main.py programa.f90 --parse

# Ver análise semântica (tabela de símbolos)
python main.py programa.f90 --semantic

# Modo verbose (mais detalhes)
python main.py programa.f90 -v
```

7.2.1. Exemplos Rápidos

```
# Hello World
python main.py tests/programs/hello.f90
cat tests/programs/hello.vm

# Condicional IF/ELSE
python main.py tests/programs/ifelse.f90

# Loop D0
python main.py tests/programs/factorial.f90
```

7.3. Como Testar Tudo

7.3.1. Rodar Todos os Testes

```
# Todos os 179 testes
python -m pytest tests/ -v

# Resultado esperado: 179 passed
```

7.3.2. Testes por Módulo

```
# Apenas Lexer (59 testes)
python -m pytest tests/test_lexer.py -v

# Apenas Parser (36 testes)
python -m pytest tests/test_parser.py -v

# Apenas Semântica (43 testes)
python -m pytest tests/test_semantic.py -v

# Apenas CodeGen (31 testes)
python -m pytest tests/test_codegen.py -v

# Apenas Integração (10 testes)
python -m pytest tests/test_integration.py -v
```

8. Conclusões e Trabalho Futuro

O desenvolvimento deste compilador permitiu implementar de forma integrada as principais fases clássicas de um compilador: análise léxica, análise sintática, análise semântica e geração de código para uma máquina virtual. O sistema desenvolvido é capaz de processar programas com declarações, expressões, estruturas condicionais, ciclos e subprogramas. A construção explícita de uma AST simplificou significativamente as fases posteriores do compilador, especialmente a análise semântica e a geração de código.

A arquitetura modular adotada contribuiu para uma separação clara entre fases, tornando o sistema mais organizado, extensível e fácil de testar. Além disso, o projeto permitiu consolidar conceitos fundamentais relacionados com gramáticas LALR(1), tabelas de símbolos, validação semântica e tradução para código executável.

Como trabalho futuro, seria interessante adicionar suporte a arrays, strings de comprimento variável e outras construções clássicas do Fortran 77, bem como introduzir mecanismos de recuperação de erros sintáticos e otimização do código gerado (e.g. eliminação de código morto, *constant folding*).

No geral, o projeto cumpriu os objetivos propostos e proporcionou uma visão prática e aprofundada do funcionamento interno de um compilador completo.